CHAPTER 4

The `class` File Format

THIS chapter describes the `class` file format of the Java Virtual Machine. Each `class` file contains the definition of a single class, interface, or module. Although a class, interface, or module need not have an external representation literally contained in a file (for instance, because the class is generated by a class loader), we will colloquially refer to any valid representation of a class, interface, or module as being in the `class` file format.

A `class` file consists of a stream of 8-bit bytes. 16-bit and 32-bit quantities are constructed by reading in two and four consecutive 8-bit bytes, respectively. Multibyte data items are always stored in big-endian order, where the high bytes come first. This chapter defines the data types `u1`, `u2`, and `u4` to represent an unsigned one-, two-, or four-byte quantity, respectively.

> In the Java SE Platform API, the `class` file format is supported by interfaces `java.io.DataInput` and `java.io.DataOutput` and classes such as `java.io.DataInputStream` and `java.io.DataOutputStream`. For example, values of the types `u1`, `u2`, and `u4` may be read by methods such as `readUnsignedByte`, `readUnsignedShort`, and `readInt` of the interface `java.io.DataInput`.

This chapter presents the `class` file format using pseudostructures written in a C-like structure notation. To avoid confusion with the fields of classes and class instances, etc., the contents of the structures describing the `class` file format are referred to as *items*. Successive items are stored in the `class` file sequentially, without padding or alignment.

Tables, consisting of zero or more variable-sized items, are used in several `class` file structures. Although we use C-like array syntax to refer to table items, the fact that tables are streams of varying-sized structures means that it is not possible to translate a table index directly to a byte offset into the table.

Where we refer to a data structure as an *array*, it consists of zero or more contiguous fixed-sized items and can be indexed like an array.

Reference to an ASCII character in this chapter should be interpreted to mean the Unicode code point corresponding to the ASCII character.

4.1 The `ClassFile` Structure

A `class` file consists of a single `ClassFile` structure:

```
ClassFile {
    u4             magic;
    u2             minor_version;
    u2             major_version;
    u2             constant_pool_count;
    cp_info        constant_pool[constant_pool_count-1];
    u2             access_flags;
    u2             this_class;
    u2             super_class;
    u2             interfaces_count;
    u2             interfaces[interfaces_count];
    u2             fields_count;
    field_info     fields[fields_count];
    u2             methods_count;
    method_info    methods[methods_count];
    u2             attributes_count;
    attribute_info attributes[attributes_count];
}
```

The items in the `ClassFile` structure are as follows:

`magic`

The `magic` item supplies the magic number identifying the `class` file format; it has the value `0xCAFEBABE`.

`minor_version, major_version`

The values of the `minor_version` and `major_version` items are the minor and major version numbers of this `class` file. Together, a major and a minor version number determine the version of the `class` file format. If a `class` file has major version number *`M`* and minor version number *`m`*, we denote the version of its `class` file format as *`M.m`*.

The `major_version` item must be a value in the range 45 through 70, inclusive. A Java Virtual Machine implementation which conforms to Java SE 26 supports precisely these major version numbers.

For a `class` file whose `major_version` is 56 or above, the `minor_version` must be 0 or 65535.

For a `class` file whose `major_version` is between 45 and 55 inclusive, the `minor_version` may be any value.

A `class` file with version number 70.65535 depends on the preview features of Java SE 26 (§1.5.1). Such a `class` file may be loaded only when the user has indicated via the host system that preview features are enabled.

A `class` file with version number M.65535, where $56 \leq M < 70$, depends on the preview features of an older release of the Java SE Platform. Such a `class` file may not be loaded, regardless of whether preview features are enabled. Instead, the `class` file may be loaded only by a Java Virtual Machine implementation that conforms to the older release.

A `class` file with any other version number does not depend on preview features, and may be loaded regardless of whether preview features are enabled.

A historical perspective is warranted on JDK use of the `class` file `minor_version`. JDK 1.0.2 supported versions 45.0 through 45.3 inclusive. JDK 1.1 supported versions 45.0 through 45.65535 inclusive. When JDK 1.2 introduced support for major version 46, the only minor version supported under that major version was 0. Later JDKs continued the practice of introducing support for a new major version (47, 48, etc.) but supporting only a minor version of 0 under the new major version. Finally, the introduction of preview features (§1.5) in Java SE 12 motivated a standard role for the minor version of the `class` file format, so JDK 12 supported minor versions of 0 *and* 65535 under major version 56. Subsequent JDKs introduce support for N.0 and N.65535 where N is the corresponding major version of the implemented Java SE Platform. For example, JDK 13 supports 57.0 and 57.65535.

`constant_pool_count`

The value of the `constant_pool_count` item is equal to the number of entries in the `constant_pool` table plus one. A `constant_pool` index is considered valid if it is greater than zero and less than `constant_pool_count`, with the exception for constants of type `long` and `double` noted in §4.4.5.

`constant_pool[]`

The `constant_pool` is a table of structures (§4.4) representing various string constants, class and interface names, field names, and other constants that are referred to within the `ClassFile` structure and its substructures. The format of each `constant_pool` table entry is indicated by its first "tag" byte.

The `constant_pool` table is indexed from 1 to `constant_pool_count` - 1.

`access_flags`

The value of the `access_flags` item is a mask of flags used to denote access permissions to and properties of this class or interface. The interpretation of each flag, when set, is specified in Table 4.1-B.

Table 4.1-B. Class access and property modifiers

Flag Name	Value	Interpretation
`ACC_PUBLIC`	0x0001	Declared `public`; may be accessed from outside its package.
`ACC_FINAL`	0x0010	Declared `final`; no subclasses allowed.
`ACC_SUPER`	0x0020	Treat superclass methods specially when invoked by the *invokespecial* instruction.
`ACC_INTERFACE`	0x0200	Is an interface, not a class.
`ACC_ABSTRACT`	0x0400	Declared `abstract`; must not be instantiated.
`ACC_SYNTHETIC`	0x1000	Declared synthetic; not present in the source code.
`ACC_ANNOTATION`	0x2000	Declared as an annotation interface.
`ACC_ENUM`	0x4000	Declared as an `enum` class.
`ACC_MODULE`	0x8000	Is a module, not a class or interface.

The `ACC_MODULE` flag indicates that this `class` file defines a module, not a class or interface. If the `ACC_MODULE` flag is set, then special rules apply to the `class` file which are given at the end of this section. If the `ACC_MODULE` flag is not set, then the rules immediately below the current paragraph apply to the `class` file.

An interface is distinguished by the `ACC_INTERFACE` flag being set. If the `ACC_INTERFACE` flag is not set, this `class` file defines a class, not an interface or module.

If the `ACC_INTERFACE` flag is set, the `ACC_ABSTRACT` flag must also be set, and the `ACC_FINAL`, `ACC_SUPER`, `ACC_ENUM`, and `ACC_MODULE` flags set must not be set.

If the `ACC_INTERFACE` flag is not set, any of the other flags in Table 4.1-B may be set except `ACC_ANNOTATION` and `ACC_MODULE`. However, such a `class` file must not have both its `ACC_FINAL` and `ACC_ABSTRACT` flags set (JLS §8.1.1.2).

The `ACC_SUPER` flag indicates which of two alternative semantics is to be expressed by the *invokespecial* instruction (§*invokespecial*) if it appears in this class or interface. Compilers to the instruction set of the Java Virtual Machine should set the `ACC_SUPER` flag. In Java SE 8 and above, the Java

Virtual Machine considers the ACC_SUPER flag to be set in every class file, regardless of the actual value of the flag in the class file and the version of the class file.

The ACC_SUPER flag exists for backward compatibility with code compiled by older compilers for the Java programming language. Prior to JDK 1.0.2, the compiler generated access_flags in which the flag now representing ACC_SUPER had no assigned meaning, and Oracle's Java Virtual Machine implementation ignored the flag if it was set.

The ACC_SYNTHETIC flag indicates that this class or interface was generated by a compiler and does not appear in source code.

An annotation interface (JLS §9.6) must have its ACC_ANNOTATION flag set. If the ACC_ANNOTATION flag is set, the ACC_INTERFACE flag must also be set.

The ACC_ENUM flag indicates that this class or its superclass is declared as an enum class (JLS §8.9).

All bits of the access_flags item not assigned in Table 4.1-B are reserved for future use. They should be set to zero in generated class files and should be ignored by Java Virtual Machine implementations.

this_class

The value of the this_class item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Class_info structure (§4.4.1) representing the class or interface defined by this class file.

super_class

For a class, the value of the super_class item either must be zero or must be a valid index into the constant_pool table. If the value of the super_class item is nonzero, the constant_pool entry at that index must be a CONSTANT_Class_info structure representing the direct superclass of the class defined by this class file. Neither the direct superclass nor any of its superclasses may have the ACC_FINAL flag set in the access_flags item of its ClassFile structure.

If the value of the super_class item is zero, then this class file must represent the class Object, the only class or interface without a direct superclass.

For an interface, the value of the super_class item must always be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Class_info structure representing the class Object.

`interfaces_count`

The value of the `interfaces_count` item gives the number of direct superinterfaces of this class or interface type.

`interfaces[]`

Each value in the `interfaces` array must be a valid index into the `constant_pool` table. The `constant_pool` entry at each value of `interfaces[`*i*`]`, where $0 \leq i <$ `interfaces_count`, must be a `CONSTANT_Class_info` structure representing an interface that is a direct superinterface of this class or interface type, in the left-to-right order given in the source for the type.

`fields_count`

The value of the `fields_count` item gives the number of `field_info` structures in the `fields` table. The `field_info` structures represent all fields, both class variables and instance variables, declared by this class or interface type.

`fields[]`

Each value in the `fields` table must be a `field_info` structure (§4.5) giving a complete description of a field in this class or interface. The `fields` table includes only those fields that are declared by this class or interface. It does not include items representing fields that are inherited from superclasses or superinterfaces.

`methods_count`

The value of the `methods_count` item gives the number of `method_info` structures in the `methods` table.

`methods[]`

Each value in the `methods` table must be a `method_info` structure (§4.6) giving a complete description of a method in this class or interface. If neither of the `ACC_NATIVE` and `ACC_ABSTRACT` flags are set in the `access_flags` item of a `method_info` structure, the Java Virtual Machine instructions implementing the method are also supplied.

The `method_info` structures represent all methods declared by this class or interface type, including instance methods, class methods, instance initialization methods (§2.9.1), and any class or interface initialization method (§2.9.2). The `methods` table does not include items representing methods that are inherited from superclasses or superinterfaces.

`attributes_count`

The value of the `attributes_count` item gives the number of attributes in the `attributes` table of this class.

`attributes[]`

Each value of the `attributes` table must be an `attribute_info` structure (§4.7).

The attributes defined by this specification as appearing in the `attributes` table of a `ClassFile` structure are listed in Table 4.7-C.

The rules concerning attributes defined to appear in the `attributes` table of a `ClassFile` structure are given in §4.7.

The rules concerning non-predefined attributes in the `attributes` table of a `ClassFile` structure are given in §4.7.1.

If the `ACC_MODULE` flag is set in the `access_flags` item, then no other flag in the `access_flags` item may be set, and the following rules apply to the rest of the `ClassFile` structure:

- `major_version`, `minor_version`: ≥ 53.0 (i.e., Java SE 9 and above)
- `this_class`: `module-info`
- `super_class`, `interfaces_count`, `fields_count`, `methods_count`: zero
- `attributes`: One `Module` attribute must be present. Except for `Module`, `ModulePackages`, `ModuleMainClass`, `InnerClasses`, `SourceFile`, `SourceDebugExtension`, `RuntimeVisibleAnnotations`, and `RuntimeInvisibleAnnotations`, none of the pre-defined attributes (§4.7) may appear.

4.2 Names

4.2.1 Binary Class and Interface Names

Class and interface names that appear in `class` file structures are always represented in a fully qualified form known as *binary names* (JLS §13.1). Such names are always represented as `CONSTANT_Utf8_info` structures (§4.4.7) and thus may be drawn, where not further constrained, from the entire Unicode codespace. Class and interface names are referenced from those `CONSTANT_NameAndType_info` structures (§4.4.6) which have such names as part of their descriptor (§4.3), and from all `CONSTANT_Class_info` structures (§4.4.1).

For historical reasons, the syntax of binary names that appear in `class` file structures differs from the syntax of binary names documented in JLS §13.1. In this internal form, the ASCII periods (`.`) that normally separate the identifiers which make up the binary name are replaced by ASCII forward slashes (`/`). The identifiers themselves must be unqualified names (§4.2.2).

> For example, the normal binary name of class `Thread` is `java.lang.Thread`. In the internal form used in descriptors in the `class` file format, a reference to the name of class `Thread` is implemented using a `CONSTANT_Utf8_info` structure representing the string `java/lang/Thread`.

4.2.2 Unqualified Names

Names of methods, fields, local variables, and formal parameters are stored as *unqualified names*. An unqualified name must contain at least one Unicode code point and must not contain any of the ASCII characters `.` `;` `[` `/` (that is, period or semicolon or left square bracket or forward slash).

Method names are further constrained so that, with the exception of the special method names `<init>` and `<clinit>` (§2.9), they must not contain the ASCII characters < or > (that is, left angle bracket or right angle bracket).

> Note that no method invocation instruction may reference `<clinit>`, and only the *invokespecial* instruction (§*invokespecial*) may reference `<init>`.

4.2.3 Module and Package Names

Module names referenced from the `Module` attribute are stored in `CONSTANT_Module_info` structures in the constant pool (§4.4.11). A `CONSTANT_Module_info` structure wraps a `CONSTANT_Utf8_info` structure that denotes the module name. Module names are not encoded in "internal form" like class and interface names, that is, the ASCII periods (`.`) that separate the identifiers in a module name are not replaced by ASCII forward slashes (`/`).

Module names may be drawn from the entire Unicode codespace, subject to the following constraints:

- A module name must not contain any code point in the range '`\u0000`' to '`\u001F`' inclusive.
- The ASCII backslash (`\`) is reserved for use as an escape character in module names. It must not appear in a module name unless it is followed by an ASCII backslash, an ASCII colon (`:`), or an ASCII at-sign (`@`). The ASCII character sequence `\\` may be used to encode a backslash in a module name.

- The ASCII colon (`:`) and at-sign (`@`) are reserved for future use in module names. They must not appear in module names unless they are escaped. The ASCII character sequences `\:` and `\@` may be used to encode a colon and an at-sign in a module name.

Package names referenced from the `Module` attribute are stored in `CONSTANT_Package_info` structures in the constant pool (§4.4.12). A `CONSTANT_Package_info` structure wraps a `CONSTANT_Utf8_info` structure that represents a package name encoded in internal form.

4.3 Descriptors

A *descriptor* is a string representing the type of a field or method. Descriptors are represented in the `class` file format using modified UTF-8 strings (§4.4.7) and thus may be drawn, where not further constrained, from the entire Unicode codespace.

4.3.1 Grammar Notation

Descriptors are specified using a grammar. The grammar is a set of productions that describe how sequences of characters can form syntactically correct descriptors of various kinds. Terminal symbols of the grammar are shown in `fixed width` font, and should be interpreted as ASCII characters. Nonterminal symbols are shown in *italic* type. The definition of a nonterminal is introduced by the name of the nonterminal being defined, followed by a colon. One or more alternative definitions for the nonterminal then follow on succeeding lines.

The syntax *{x}* on the right-hand side of a production denotes zero or more occurrences of *x*.

The phrase *(one of)* on the right-hand side of a production signifies that each of the terminal symbols on the following line or lines is an alternative definition.

4.3.2 Field Descriptors

A *field descriptor* represents the type of a field, parameter, local variable, or value.

FieldDescriptor:
  FieldType

FieldType:
BaseType
ClassType
ArrayType

BaseType:
(one of)
`B C D F I J S Z`

ClassType:
`L` *ClassName* `;`

ArrayType:
`[` *ComponentType*

ComponentType:
FieldType

ClassName represents a binary class or interface name encoded in internal form (§4.2.1).

A field descriptor *mentions* a class or interface name if the name appears as a *ClassName* in the descriptor. This includes a *ClassName* nested in the *ComponentType* of an *ArrayType*.

The interpretation of field descriptors as types is shown in Table 4.3-A. See §2.2, §2.3, and §2.4 for the meaning of these types.

A field descriptor representing an array type is valid only if it represents a type with 255 or fewer dimensions.

Table 4.3-A. Interpretation of field descriptors

***FieldType* term**	**Type**
`B`	`byte`
`C`	`char`
`D`	`double`
`F`	`float`
`I`	`int`
`J`	`long`
`L` *ClassName* `;`	Named class or interface type
`S`	`short`
`Z`	`boolean`
`[` *ComponentType*	Array of given component type

The field descriptor of an instance variable of type `int` is simply `I`.

The field descriptor of an instance variable of type `Object` is `Ljava/lang/Object;`. Note that the internal form of the binary name for class `Object` is used.

The field descriptor of an instance variable of the multidimensional array type `double[][][]` is `[[[D`.

4.3.3 Method Descriptors

A *method descriptor* contains zero or more *parameter descriptors*, representing the types of parameters that the method takes, and a *return descriptor*, representing the type of the value (if any) that the method returns.

MethodDescriptor:
`(` *{ParameterDescriptor}* `)` *ReturnDescriptor*

ParameterDescriptor:
FieldType

ReturnDescriptor:
FieldType
VoidDescriptor

VoidDescriptor:
V

The character V indicates that the method returns no value (its result is void).

A method descriptor *mentions* a class or interface name if the name appears as a *ClassName* in the *FieldType* of a parameter descriptor or return descriptor.

The method descriptor for the method:

```
Object m(int i, double d, Thread t) {...}
```

is:

```
(IDLjava/lang/Thread;)Ljava/lang/Object;
```

Note that the internal forms of the binary names of Thread and Object are used.

A method descriptor is valid only if it represents method parameters with a total length of 255 or less, where that length includes the contribution for this in the case of instance or interface method invocations. The total length is calculated by summing the contributions of the individual parameters, where a parameter of type long or double contributes two units to the length and a parameter of any other type contributes one unit.

A method descriptor is the same whether the method it describes is a class method or an instance method. Although an instance method is passed this, a reference to the object on which the method is being invoked, in addition to its intended arguments, that fact is not reflected in the method descriptor. The reference to this is passed implicitly by the Java Virtual Machine instructions which invoke instance methods (§2.6.1, §4.11).

4.4 The Constant Pool

Java Virtual Machine instructions do not rely on the run-time layout of classes, interfaces, class instances, or arrays. Instead, instructions refer to symbolic information in the constant_pool table.

All constant_pool table entries have the following general format:

```
cp_info {
    u1 tag;
    u1 info[];
}
```

Each entry in the `constant_pool` table must begin with a 1-byte tag indicating the kind of constant denoted by the entry. There are 17 kinds of constant, listed in Table 4.4-A with their corresponding tags, and ordered by their section number in this chapter. Each tag byte must be followed by two or more bytes giving information about the specific constant. The format of the additional information depends on the tag byte, that is, the content of the `info` array varies with the value of `tag`.

Table 4.4-A. Constant pool tags (by section)

Constant Kind	Tag	Section
`CONSTANT_Class`	7	§4.4.1
`CONSTANT_Fieldref`	9	§4.4.2
`CONSTANT_Methodref`	10	§4.4.2
`CONSTANT_InterfaceMethodref`	11	§4.4.2
`CONSTANT_String`	8	§4.4.3
`CONSTANT_Integer`	3	§4.4.4
`CONSTANT_Float`	4	§4.4.4
`CONSTANT_Long`	5	§4.4.5
`CONSTANT_Double`	6	§4.4.5
`CONSTANT_NameAndType`	12	§4.4.6
`CONSTANT_Utf8`	1	§4.4.7
`CONSTANT_MethodHandle`	15	§4.4.8
`CONSTANT_MethodType`	16	§4.4.9
`CONSTANT_Dynamic`	17	§4.4.10
`CONSTANT_InvokeDynamic`	18	§4.4.10
`CONSTANT_Module`	19	§4.4.11
`CONSTANT_Package`	20	§4.4.12

In a `class` file whose version number is *v*, each entry in the `constant_pool` table must have a tag that was first defined in version *v* or earlier of the `class` file format (§4.1). That is, each entry must denote a kind of constant that is approved for use in the `class` file. Table 4.4-B lists each tag with the first version of the `class` file format in which it was defined. Also shown is the version of the Java SE Platform which introduced that version of the `class` file format.

Table 4.4-B. Constant pool tags (by tag)

Constant Kind	Tag	class file format	Java SE
CONSTANT_Utf8	1	45.3	1.0.2
CONSTANT_Integer	3	45.3	1.0.2
CONSTANT_Float	4	45.3	1.0.2
CONSTANT_Long	5	45.3	1.0.2
CONSTANT_Double	6	45.3	1.0.2
CONSTANT_Class	7	45.3	1.0.2
CONSTANT_String	8	45.3	1.0.2
CONSTANT_Fieldref	9	45.3	1.0.2
CONSTANT_Methodref	10	45.3	1.0.2
CONSTANT_InterfaceMethodref	11	45.3	1.0.2
CONSTANT_NameAndType	12	45.3	1.0.2
CONSTANT_MethodHandle	15	51.0	7
CONSTANT_MethodType	16	51.0	7
CONSTANT_Dynamic	17	55.0	11
CONSTANT_InvokeDynamic	18	51.0	7
CONSTANT_Module	19	53.0	9
CONSTANT_Package	20	53.0	9

Some entries in the constant_pool table are *loadable* because they represent entities that can be pushed onto the stack at run time to enable further computation. In a class file whose version number is *v*, an entry in the constant_pool table is loadable if it has a tag that was first deemed to be loadable in version *v* or earlier of the class file format. Table 4.4-C lists each tag with the first version of the class file format in which it was deemed to be loadable. Also shown is the version of the Java SE Platform which introduced that version of the class file format.

In every case except CONSTANT_Class, a tag was first deemed to be loadable in the same version of the class file format that first defined the tag.

Table 4.4-C. Loadable constant pool tags

Constant Kind	Tag	class file format	Java SE
CONSTANT_Integer	3	45.3	1.0.2
CONSTANT_Float	4	45.3	1.0.2
CONSTANT_Long	5	45.3	1.0.2
CONSTANT_Double	6	45.3	1.0.2
CONSTANT_Class	7	49.0	5.0
CONSTANT_String	8	45.3	1.0.2
CONSTANT_MethodHandle	15	51.0	7
CONSTANT_MethodType	16	51.0	7
CONSTANT_Dynamic	17	55.0	11

4.4.1 The CONSTANT_Class_info Structure

The CONSTANT_Class_info structure is used to represent a class or an interface:

```
CONSTANT_Class_info {
    u1 tag;
    u2 name_index;
}
```

The items of the CONSTANT_Class_info structure are as follows:

tag

The tag item has the value CONSTANT_Class (7).

name_index

The value of the name_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Utf8_info structure (§4.4.7) representing a valid binary class or interface name encoded in internal form (§4.2.1).

Because arrays are objects, the opcodes *anewarray* and *multianewarray* - but not the opcode *new* - can reference array "classes" via CONSTANT_Class_info structures in the constant_pool table. For such array classes, the name of the class is the descriptor of the array type (§4.3.2).

For example, the class name representing the two-dimensional array type int[][] is [[I, while the class name representing the type Thread[] is [Ljava/lang/Thread;.

An array type descriptor is valid only if it represents 255 or fewer dimensions.

4.4.2 The CONSTANT_Fieldref_info, CONSTANT_Methodref_info, and CONSTANT_InterfaceMethodref_info Structures

Fields, methods, and interface methods are represented by similar structures:

```
CONSTANT_Fieldref_info {
    u1 tag;
    u2 class_index;
    u2 name_and_type_index;
}

CONSTANT_Methodref_info {
    u1 tag;
    u2 class_index;
    u2 name_and_type_index;
}

CONSTANT_InterfaceMethodref_info {
    u1 tag;
    u2 class_index;
    u2 name_and_type_index;
}
```

The items of these structures are as follows:

`tag`

The `tag` item of a `CONSTANT_Fieldref_info` structure has the value `CONSTANT_Fieldref` (9).

The `tag` item of a `CONSTANT_Methodref_info` structure has the value `CONSTANT_Methodref` (10).

The `tag` item of a `CONSTANT_InterfaceMethodref_info` structure has the value `CONSTANT_InterfaceMethodref` (11).

`class_index`

The value of the `class_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Class_info` structure (§4.4.1) representing a class or interface type that has the field or method as a member.

In a `CONSTANT_Fieldref_info` structure, the `class_index` item may be either a class type or an interface type.

In a `CONSTANT_Methodref_info` structure, the `class_index` item should be a class type, not an interface type.

In a `CONSTANT_InterfaceMethodref_info` structure, the `class_index` item should be an interface type, not a class type.

`name_and_type_index`

The value of the `name_and_type_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_NameAndType_info` structure (§4.4.6). This `constant_pool` entry indicates the name and descriptor of the field or method.

In a `CONSTANT_Fieldref_info` structure, the indicated descriptor must be a field descriptor (§4.3.2). Otherwise, the indicated descriptor must be a method descriptor (§4.3.3).

If the name of the method in a `CONSTANT_Methodref_info` structure begins with a '`<`' ('`\u003c`'), then the name must be the special name `<init>`, representing an instance initialization method (§2.9.1). The return type of such a method must be `void`.

4.4.3 The `CONSTANT_String_info` Structure

The `CONSTANT_String_info` structure is used to represent constant objects of the type `String`:

```
CONSTANT_String_info {
    u1 tag;
    u2 string_index;
}
```

The items of the `CONSTANT_String_info` structure are as follows:

`tag`

The `tag` item has the value `CONSTANT_String` (8).

`string_index`

The value of the `string_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing the sequence of Unicode code points to which the `String` object is to be initialized.

4.4.4 The `CONSTANT_Integer_info` and `CONSTANT_Float_info` Structures

The `CONSTANT_Integer_info` and `CONSTANT_Float_info` structures represent 4-byte numeric (`int` and `float`) constants:

```
CONSTANT_Integer_info {
    u1 tag;
    u4 bytes;
}

CONSTANT_Float_info {
    u1 tag;
    u4 bytes;
}
```

The items of these structures are as follows:

`tag`

The `tag` item of the `CONSTANT_Integer_info` structure has the value `CONSTANT_Integer` (3).

The `tag` item of the `CONSTANT_Float_info` structure has the value `CONSTANT_Float` (4).

`bytes`

The `bytes` item of the `CONSTANT_Integer_info` structure represents the value of the `int` constant. The bytes of the value are stored in big-endian (high byte first) order.

The `bytes` item of the `CONSTANT_Float_info` structure represents the value of the `float` constant in IEEE 754 binary32 floating-point format (§2.3.2). The bytes of the item are stored in big-endian (high byte first) order.

The value represented by the `CONSTANT_Float_info` structure is determined as follows. The bytes of the value are first converted into an `int` constant *bits*. Then:

- If *bits* is `0x7f800000`, the `float` value will be positive infinity.
- If *bits* is `0xff800000`, the `float` value will be negative infinity.
- If *bits* is in the range `0x7f800001` through `0x7fffffff` or in the range `0xff800001` through `0xffffffff`, the `float` value will be NaN.
- In all other cases, let `s`, `e`, and `m` be three values that might be computed from *bits*:

```
int s = ((bits >> 31) == 0) ? 1 : -1;
int e = ((bits >> 23) & 0xff);
int m = (e == 0) ?
          (bits & 0x7fffff) << 1 :
          (bits & 0x7fffff) | 0x800000;
```

Then the `float` value equals the result of the mathematical expression $s \cdot m \cdot 2^{e-150}$.

4.4.5 The `CONSTANT_Long_info` and `CONSTANT_Double_info` Structures

The `CONSTANT_Long_info` and `CONSTANT_Double_info` represent 8-byte numeric (`long` and `double`) constants:

```
CONSTANT_Long_info {
    u1 tag;
    u4 high_bytes;
    u4 low_bytes;
}

CONSTANT_Double_info {
    u1 tag;
    u4 high_bytes;
    u4 low_bytes;
}
```

All 8-byte constants take up two entries in the `constant_pool` table of the `class` file. If a `CONSTANT_Long_info` or `CONSTANT_Double_info` structure is the entry at index *n* in the `constant_pool` table, then the next usable entry in the table is located at index *n*+2. The `constant_pool` index *n*+1 must be valid but is considered unusable.

In retrospect, making 8-byte constants take two constant pool entries was a poor choice.

The items of these structures are as follows:

`tag`

The `tag` item of the `CONSTANT_Long_info` structure has the value `CONSTANT_Long` (5).

The `tag` item of the `CONSTANT_Double_info` structure has the value `CONSTANT_Double` (6).

`high_bytes, low_bytes`

The unsigned `high_bytes` and `low_bytes` items of the `CONSTANT_Long_info` structure together represent the value of the `long` constant

```
((long) high_bytes << 32) + low_bytes
```

where the bytes of each of `high_bytes` and `low_bytes` are stored in big-endian (high byte first) order.

The `high_bytes` and `low_bytes` items of the `CONSTANT_Double_info` structure together represent the `double` value in IEEE 754 binary64 floating-point format (§2.3.2). The bytes of each item are stored in big-endian (high byte first) order.

The value represented by the `CONSTANT_Double_info` structure is determined as follows. The `high_bytes` and `low_bytes` items are converted into the `long` constant *bits*, which is equal to

```
((long) high_bytes << 32) + low_bytes
```

Then:

- If *bits* is `0x7ff0000000000000L`, the `double` value will be positive infinity.
- If *bits* is `0xfff0000000000000L`, the `double` value will be negative infinity.
- If *bits* is in the range `0x7ff0000000000001L` through `0x7fffffffffffffffL` or in the range `0xfff0000000000001L` through `0xffffffffffffffffL`, the double value will be NaN.
- In all other cases, let `s`, `e`, and `m` be three values that might be computed from *bits*:

```
int s = ((bits >> 63) == 0) ? 1 : -1;
int e = (int)((bits >> 52) & 0x7ffL);
long m = (e == 0) ?
            (bits & 0xfffffffffffffL) << 1 :
            (bits & 0xfffffffffffffL) | 0x10000000000000L;
```

Then the floating-point value equals the `double` value of the mathematical expression $s \cdot m \cdot 2^{e-1075}$.

4.4.6 The `CONSTANT_NameAndType_info` Structure

The `CONSTANT_NameAndType_info` structure is used to represent a field or method, without indicating which class or interface type it belongs to:

```
CONSTANT_NameAndType_info {
    u1 tag;
    u2 name_index;
    u2 descriptor_index;
}
```

The items of the `CONSTANT_NameAndType_info` structure are as follows:

`tag`

The `tag` item has the value `CONSTANT_NameAndType` (12).

`name_index`

The value of the `name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing either a valid unqualified name denoting a field or method (§4.2.2), or the special method name `<init>` (§2.9.1).

`descriptor_index`

The value of the `descriptor_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing a valid field descriptor or method descriptor (§4.3.2, §4.3.3).

4.4.7 The `CONSTANT_Utf8_info` Structure

The `CONSTANT_Utf8_info` structure is used to represent constant string values:

```
CONSTANT_Utf8_info {
    u1 tag;
    u2 length;
    u1 bytes[length];
}
```

The items of the `CONSTANT_Utf8_info` structure are as follows:

`tag`

The `tag` item has the value `CONSTANT_Utf8` (1).

`length`

The value of the `length` item gives the number of bytes in the `bytes` array (not the length of the resulting string).

`bytes[]`

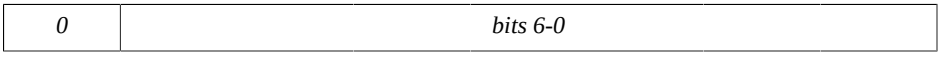
The `bytes` array contains the bytes of the string.

No byte may have the value `(byte)0`.

No byte may lie in the range `(byte)0xf0` to `(byte)0xff`.

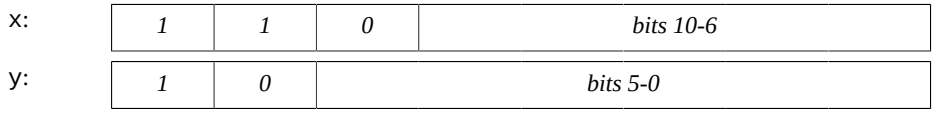
String content is encoded in modified UTF-8. Modified UTF-8 strings are encoded so that code point sequences that contain only non-null ASCII characters can be represented using only 1 byte per code point, but all code points in the Unicode codespace can be represented. Modified UTF-8 strings are not null-terminated. The encoding is as follows:

- Code points in the range '\u0001' to '\u007F' are represented by a single byte:



  The 7 bits of data in the byte give the value of the code point represented.

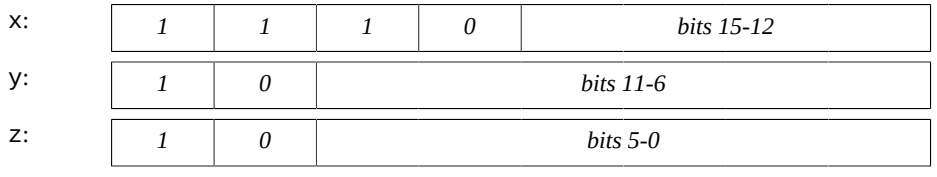
- The null code point ('\u0000') and code points in the range '\u0080' to '\u07FF' are represented by a pair of bytes x and y :



  The two bytes represent the code point with the value:

```
((x & 0x1f) << 6) + (y & 0x3f)
```

- Code points in the range '\u0800' to '\uFFFF' are represented by 3 bytes x, y, and z :



  The three bytes represent the code point with the value:

```
((x & 0xf) << 12) + ((y & 0x3f) << 6) + (z & 0x3f)
```

- Characters with code points above U+FFFF (so-called *supplementary characters*) are represented by separately encoding the two surrogate code units of their UTF-16 representation. Each of the surrogate code units is represented by

three bytes. This means supplementary characters are represented by six bytes, `u`, `v`, `w`, `x`, `y`, and `z`:

u:	*1*	*1*	*1*	*0*	*1*	*1*	*0*	*1*
v:	*1*	*0*	*1*	*0*	*(bits 20-16)-1*			
w:	*1*	*0*	*bits 15-10*					
x:	*1*	*1*	*1*	*0*	*1*	*1*	*0*	*1*
y:	*1*	*0*	*1*	*1*	*bits 9-6*			
z:	*1*	*0*	*bits 5-0*					

The six bytes represent the code point with the value:

```
0x10000 + ((v & 0x0f) << 16) + ((w & 0x3f) << 10) +
((y & 0x0f) << 6) + (z & 0x3f)
```

The bytes of multibyte characters are stored in the `class` file in big-endian (high byte first) order.

There are two differences between this format and the "standard" UTF-8 format. First, the null character `(char)0` is encoded using the 2-byte format rather than the 1-byte format, so that modified UTF-8 strings never have embedded nulls. Second, only the 1-byte, 2-byte, and 3-byte formats of standard UTF-8 are used. The Java Virtual Machine does not recognize the four-byte format of standard UTF-8; it uses its own two-times-three-byte format instead.

> For more information regarding the standard UTF-8 format, see Section 3.9 *Unicode Encoding Forms* of *The Unicode Standard, Version 17.0.0*.

4.4.8 The `CONSTANT_MethodHandle_info` Structure

The `CONSTANT_MethodHandle_info` structure is used to represent a method handle:

```
CONSTANT_MethodHandle_info {
    u1 tag;
    u1 reference_kind;
    u2 reference_index;
}
```

The items of the `CONSTANT_MethodHandle_info` structure are the following:

`tag`

The `tag` item has the value `CONSTANT_MethodHandle` (15).

`reference_kind`

The value of the `reference_kind` item must be in the range 1 to 9. The value denotes the *kind* of this method handle, which characterizes its bytecode behavior (§5.4.3.5).

`reference_index`

The value of the `reference_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be as follows:

- If the value of the `reference_kind` item is 1 (`REF_getField`), 2 (`REF_getStatic`), 3 (`REF_putField`), or 4 (`REF_putStatic`), then the `constant_pool` entry at that index must be a `CONSTANT_Fieldref_info` structure (§4.4.2) representing a field for which a method handle is to be created.
- If the value of the `reference_kind` item is 5 (`REF_invokeVirtual`) or 8 (`REF_newInvokeSpecial`), then the `constant_pool` entry at that index must be a `CONSTANT_Methodref_info` structure (§4.4.2) representing a class's method or constructor (§2.9.1) for which a method handle is to be created.
- If the value of the `reference_kind` item is 6 (`REF_invokeStatic`) or 7 (`REF_invokeSpecial`), then if the `class` file version number is less than 52.0, the `constant_pool` entry at that index must be a `CONSTANT_Methodref_info` structure representing a class's method for which a method handle is to be created; if the `class` file version number is 52.0 or above, the `constant_pool` entry at that index must be either a `CONSTANT_Methodref_info` structure or a `CONSTANT_InterfaceMethodref_info` structure (§4.4.2) representing a class's or interface's method for which a method handle is to be created.
- If the value of the `reference_kind` item is 9 (`REF_invokeInterface`), then the `constant_pool` entry at that index must be a `CONSTANT_InterfaceMethodref_info` structure representing an interface's method for which a method handle is to be created.

If the value of the `reference_kind` item is 5 (`REF_invokeVirtual`), 6 (`REF_invokeStatic`), 7 (`REF_invokeSpecial`), or 9 (`REF_invokeInterface`), the name of the method represented by a `CONSTANT_Methodref_info` structure or a `CONSTANT_InterfaceMethodref_info` structure must not be `<init>` or `<clinit>`.

If the value is 8 (`REF_newInvokeSpecial`), the name of the method represented by a `CONSTANT_Methodref_info` structure must be `<init>`.

4.4.9 The CONSTANT_MethodType_info Structure

The CONSTANT_MethodType_info structure is used to represent a method type:

```
CONSTANT_MethodType_info {
    u1 tag;
    u2 descriptor_index;
}
```

The items of the CONSTANT_MethodType_info structure are as follows:

tag

The tag item has the value CONSTANT_MethodType (16).

descriptor_index

The value of the descriptor_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Utf8_info structure (§4.4.7) representing a method descriptor (§4.3.3).

4.4.10 The CONSTANT_Dynamic_info and CONSTANT_InvokeDynamic_info Structures

Most structures in the constant_pool table represent entities directly, by combining names, descriptors, and values recorded statically in the table. In contrast, the CONSTANT_Dynamic_info and CONSTANT_InvokeDynamic_info structures represent entities indirectly, by pointing to code which computes an entity dynamically. The code, called a *bootstrap method*, is invoked by the Java Virtual Machine during resolution of symbolic references derived from these structures (§5.1, §5.4.3.6). Each structure specifies a bootstrap method as well as an auxiliary name and type that characterize the entity to be computed. In more detail:

- The CONSTANT_Dynamic_info structure is used to represent a *dynamically-computed constant*, an arbitrary value that is produced by invocation of a bootstrap method in the course of an *ldc* instruction (§*ldc*), among others. The auxiliary type specified by the structure constrains the type of the dynamically-computed constant.
- The CONSTANT_InvokeDynamic_info structure is used to represent a *dynamically-computed call site*, an instance of java.lang.invoke.CallSite that is produced by invocation of a bootstrap method in the course of an *invokedynamic* instruction (§*invokedynamic*). The auxiliary type specified by the structure constrains the method type of the dynamically-computed call site.

```
CONSTANT_Dynamic_info {
    u1 tag;
    u2 bootstrap_method_attr_index;
    u2 name_and_type_index;
}

CONSTANT_InvokeDynamic_info {
    u1 tag;
    u2 bootstrap_method_attr_index;
    u2 name_and_type_index;
}
```

The items of these structures are as follows:

`tag`

The `tag` item of a `CONSTANT_Dynamic_info` structure has the value `CONSTANT_Dynamic` (17).

The `tag` item of a `CONSTANT_InvokeDynamic_info` structure has the value `CONSTANT_InvokeDynamic` (18).

`bootstrap_method_attr_index`

The value of the `bootstrap_method_attr_index` item must be a valid index into the `bootstrap_methods` array of the bootstrap method table of this `class` file (§4.7.23).

`CONSTANT_Dynamic_info` structures are unique in that they are syntactically allowed to refer to themselves via the bootstrap method table. Rather than mandating that such cycles are detected when classes are loaded (a potentially expensive check), we permit cycles initially but mandate a failure at resolution (§5.4.3.6).

`name_and_type_index`

The value of the `name_and_type_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_NameAndType_info` structure (§4.4.6). This `constant_pool` entry indicates a name and descriptor.

In a `CONSTANT_Dynamic_info` structure, the indicated descriptor must be a field descriptor (§4.3.2).

In a `CONSTANT_InvokeDynamic_info` structure, the indicated descriptor must be a method descriptor (§4.3.3).

4.4.11 The `CONSTANT_Module_info` Structure

The `CONSTANT_Module_info` structure is used to represent a module:

```
CONSTANT_Module_info {
    u1 tag;
    u2 name_index;
}
```

The items of the `CONSTANT_Module_info` structure are as follows:

`tag`

The `tag` item has the value `CONSTANT_Module` (19).

`name_index`

The value of the `name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing a valid module name (§4.2.3).

A `CONSTANT_Module_info` structure is permitted only in the constant pool of a `class` file that declares a module, that is, a `ClassFile` structure where the `access_flags` item has the `ACC_MODULE` flag set. In all other `class` files, a `CONSTANT_Module_info` structure is illegal.

4.4.12 The `CONSTANT_Package_info` Structure

The `CONSTANT_Package_info` structure is used to represent a package exported or opened by a module:

```
CONSTANT_Package_info {
    u1 tag;
    u2 name_index;
}
```

The items of the `CONSTANT_Package_info` structure are as follows:

`tag`

The `tag` item has the value `CONSTANT_Package` (20).

`name_index`

The value of the `name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing a valid package name encoded in internal form (§4.2.3).

A `CONSTANT_Package_info` structure is permitted only in the constant pool of a `class` file that declares a module, that is, a `ClassFile` structure where the

access_flags item has the ACC_MODULE flag set. In all other class files, a CONSTANT_Package_info structure is illegal.

4.5 Fields

Each field is described by a field_info structure.

No two fields in one class file may have the same name and descriptor (§4.3.2).

The structure has the following format:

```
field_info {
    u2             access_flags;
    u2             name_index;
    u2             descriptor_index;
    u2             attributes_count;
    attribute_info attributes[attributes_count];
}
```

The items of the field_info structure are as follows:

access_flags

The value of the access_flags item is a mask of flags used to denote access permission to and properties of this field. The interpretation of each flag, when set, is specified in Table 4.5-A.

Table 4.5-A. Field access and property flags

Flag Name	Value	Interpretation
ACC_PUBLIC	0x0001	Declared public; may be accessed from outside its package.
ACC_PRIVATE	0x0002	Declared private; accessible only within the defining class and other classes belonging to the same nest (§5.4.4).
ACC_PROTECTED	0x0004	Declared protected; may be accessed within subclasses.
ACC_STATIC	0x0008	Declared static.
ACC_FINAL	0x0010	Declared final; never directly assigned to after object construction (JLS §17.5).
ACC_VOLATILE	0x0040	Declared volatile; cannot be cached.
ACC_TRANSIENT	0x0080	Declared transient; not written or read by a persistent object manager.
ACC_SYNTHETIC	0x1000	Declared synthetic; not present in the source code.
ACC_ENUM	0x4000	Declared as an element of an enum class.

Fields of classes may set any of the flags in Table 4.5-A. However, each field of a class may have at most one of its ACC_PUBLIC, ACC_PRIVATE, and ACC_PROTECTED flags set (JLS §8.3.1), and must not have both its ACC_FINAL and ACC_VOLATILE flags set (JLS §8.3.1.4).

Fields of interfaces must have their ACC_PUBLIC, ACC_STATIC, and ACC_FINAL flags set; they may have their ACC_SYNTHETIC flag set and must not have any of the other flags in Table 4.5-A set (JLS §9.3).

The ACC_SYNTHETIC flag indicates that this field was generated by a compiler and does not appear in source code.

The ACC_ENUM flag indicates that this field is used to hold an element of an enum class (JLS §8.9).

All bits of the access_flags item not assigned in Table 4.5-A are reserved for future use. They should be set to zero in generated class files and should be ignored by Java Virtual Machine implementations.

name_index

The value of the name_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a

`CONSTANT_Utf8_info` structure (§4.4.7) which represents a valid unqualified name denoting a field (§4.2.2).

`descriptor_index`

The value of the `descriptor_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) which represents a valid field descriptor (§4.3.2).

`attributes_count`

The value of the `attributes_count` item indicates the number of additional attributes of this field.

`attributes[]`

Each value of the `attributes` table must be an `attribute_info` structure (§4.7).

A field can have any number of optional attributes associated with it.

The attributes defined by this specification as appearing in the `attributes` table of a `field_info` structure are listed in Table 4.7-C.

The rules concerning attributes defined to appear in the `attributes` table of a `field_info` structure are given in §4.7.

The rules concerning non-predefined attributes in the `attributes` table of a `field_info` structure are given in §4.7.1.

4.6 Methods

Each method, including each instance initialization method (§2.9.1) and the class or interface initialization method (§2.9.2), is described by a `method_info` structure.

No two methods in one `class` file may have the same name and descriptor (§4.3.3).

The structure has the following format:

```
method_info {
    u2             access_flags;
    u2             name_index;
    u2             descriptor_index;
    u2             attributes_count;
    attribute_info attributes[attributes_count];
}
```

The items of the `method_info` structure are as follows:

`access_flags`

The value of the `access_flags` item is a mask of flags used to denote access permission to and properties of this method. The interpretation of each flag, when set, is specified in Table 4.6-A.

Table 4.6-A. Method access and property flags

Flag Name	Value	Interpretation
`ACC_PUBLIC`	0x0001	Declared `public`; may be accessed from outside its package.
`ACC_PRIVATE`	0x0002	Declared `private`; accessible only within the defining class and other classes belonging to the same nest (§5.4.4).
`ACC_PROTECTED`	0x0004	Declared `protected`; may be accessed within subclasses.
`ACC_STATIC`	0x0008	Declared `static`.
`ACC_FINAL`	0x0010	Declared `final`; must not be overridden (§5.4.5).
`ACC_SYNCHRONIZED`	0x0020	Declared `synchronized`; invocation is wrapped by a monitor use.
`ACC_BRIDGE`	0x0040	A bridge method, generated by the compiler.
`ACC_VARARGS`	0x0080	Declared with variable number of arguments.
`ACC_NATIVE`	0x0100	Declared `native`; implemented in a language other than the Java programming language.
`ACC_ABSTRACT`	0x0400	Declared `abstract`; no implementation is provided.
`ACC_STRICT`	0x0800	In a `class` file whose major version number is at least 46 and at most 60: Declared `strictfp`.
`ACC_SYNTHETIC`	0x1000	Declared synthetic; not present in the source code.

The value 0x0800 is interpreted as the `ACC_STRICT` flag only in a `class` file whose major version number is at least 46 and at most 60. For methods in such a `class` file, the rules below determine whether the `ACC_STRICT` flag may be set in combination with other flags. (Setting the `ACC_STRICT` flag constrained a method's floating-point instructions in Java SE 1.2 through 16 (§2.8).) For methods in a `class` file whose major version number is less than 46 or greater than 60, the value 0x0800 is not interpreted as the `ACC_STRICT` flag, but rather is unassigned; it is not meaningful to "set the `ACC_STRICT` flag" in such a `class` file.

Methods of classes may have any of the flags in Table 4.6-A set. However, each method of a class may have at most one of its ACC_PUBLIC, ACC_PRIVATE, and ACC_PROTECTED flags set (JLS §8.4.3).

Methods of interfaces may have any of the flags in Table 4.6-A set except ACC_PROTECTED, ACC_FINAL, ACC_SYNCHRONIZED, and ACC_NATIVE (JLS §9.4). In a class file whose version number is less than 52.0, each method of an interface must have its ACC_PUBLIC and ACC_ABSTRACT flags set; in a class file whose version number is 52.0 or above, each method of an interface must have exactly one of its ACC_PUBLIC and ACC_PRIVATE flags set.

If a method of a class or interface has its ACC_ABSTRACT flag set, it must not have any of its ACC_PRIVATE, ACC_STATIC, ACC_FINAL, ACC_SYNCHRONIZED, or ACC_NATIVE flags set, nor (in a class file whose major version number is at least 46 and at most 60) have its ACC_STRICT flag set.

An instance initialization method (§2.9.1) may have at most one of its ACC_PUBLIC, ACC_PRIVATE, and ACC_PROTECTED flags set, and may also have its ACC_VARARGS and ACC_SYNTHETIC flags set, and may also (in a class file whose major version number is at least 46 and at most 60) have its ACC_STRICT flag set, but must not have any of the other flags in Table 4.6-A set.

In a class file whose version number is 51.0 or above, a method whose name is <clinit> must have its ACC_STATIC flag set.

A class or interface initialization method (§2.9.2) is called implicitly by the Java Virtual Machine. The value of its access_flags item is ignored except for the setting of the ACC_STATIC flag and (in a class file whose major version number is at least 46 and at most 60) the ACC_STRICT flag, and the method is exempt from the preceding rules about legal combinations of flags.

The ACC_BRIDGE flag is used to indicate a bridge method generated by a compiler for the Java programming language.

The ACC_VARARGS flag indicates that this method takes a variable number of arguments at the source code level. A method declared to take a variable number of arguments must be compiled with the ACC_VARARGS flag set to 1. All other methods must be compiled with the ACC_VARARGS flag set to 0.

The ACC_SYNTHETIC flag indicates that this method was generated by a compiler and does not appear in source code, unless it is one of the methods named in §4.7.8.

All bits of the access_flags item not assigned in Table 4.6-A are reserved for future use. (This includes the bit corresponding to 0x0800 in a class file

whose major version number is less than 46 or greater than 60.) They should be set to zero in generated `class` files and should be ignored by Java Virtual Machine implementations.

`name_index`

The value of the `name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing either a valid unqualified name denoting a method (§4.2.2), or (if this method is in a class rather than an interface) the special method name `<init>`, or the special method name `<clinit>`.

`descriptor_index`

The value of the `descriptor_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure representing a valid method descriptor (§4.3.3). Furthermore:

- If this method is in a class rather than an interface, and the name of the method is `<init>`, then the descriptor must denote a `void` method.
- If the name of the method is `<clinit>`, then the descriptor must denote a `void` method, and, in a `class` file whose version number is 51.0 or above, a method that takes no arguments.

A future edition of this specification may require that the last parameter descriptor of the method descriptor is an array type if the `ACC_VARARGS` flag is set in the `access_flags` item.

`attributes_count`

The value of the `attributes_count` item indicates the number of additional attributes of this method.

`attributes[]`

Each value of the `attributes` table must be an `attribute_info` structure (§4.7).

A method can have any number of optional attributes associated with it.

The attributes defined by this specification as appearing in the `attributes` table of a `method_info` structure are listed in Table 4.7-C.

The rules concerning attributes defined to appear in the `attributes` table of a `method_info` structure are given in §4.7.